

PATRÓN DE DISEÑO: PUENTE (BRIDGE)

Capítulo 24 — Patrones de Diseño en Java

Traducción técnica al español

© 2004 CRC Press LLC — Traducción al español

1. Descripción

Este patrón fue descrito previamente en GoF95.

El Patrón Puente promueve la separación de la interfaz de una abstracción de su implementación. En términos generales, el término abstracción hace referencia al proceso de identificar el conjunto de atributos y comportamientos de un objeto que son específicos para un propósito determinado. Esta visión específica de un objeto puede diseñarse como un objeto separado, omitiendo los atributos y comportamientos irrelevantes. El objeto resultante en sí mismo puede denominarse abstracción. Nótese que un objeto dado puede tener más de una abstracción, cada una con un uso distinto.

Una abstracción determinada puede tener una o más implementaciones para su comportamiento. En términos de implementación, una abstracción puede diseñarse como una interfaz con uno o más implementadores concretos.

En la jerarquía de clases sin el patrón, la interfaz Abstraction declara un conjunto de métodos que representan el resultado de abstraer la funcionalidad común de diferentes objetos. Tanto `Implementer_1` como `Implementer_2` son implementadores del conjunto de Abstraction. Sin embargo, este enfoque presenta dos limitaciones importantes:

1. Cuando existe la necesidad de subclasificar la jerarquía por algún otro propósito, esto puede llevar a un número exponencial de subclases y pronto se tendrá una jerarquía de clases explosiva.
2. Tanto la interfaz de la abstracción como su implementación están vinculadas entre sí y, por lo tanto, no pueden variar de manera independiente sin afectarse mutuamente.

Usando el Patrón Puente, se puede lograr un diseño de abstracción más eficiente y manejable. El diseño de una abstracción usando el Patrón Puente separa sus interfaces de sus implementaciones. Al aplicar el Patrón Puente, las interfaces y las implementaciones de una abstracción pueden colocarse en dos jerarquías de clases separadas.

La abstracción mantiene una referencia de objeto del tipo `Implementer`. Una aplicación cliente puede elegir el tipo de abstracción deseado de la jerarquía de clases de interfaz, y el objeto de abstracción puede configurarse luego con una instancia de un implementador de la jerarquía de implementadores. Esta capacidad de combinar abstracciones e implementaciones

dinámicamente puede ser muy útil para extender la funcionalidad sin recurrir a la subclasificación. Cuando un objeto cliente invoca un método sobre el objeto Abstraction, este reenvía la llamada al objeto Implementer que contiene, pudiendo ofrecer algún preprocesamiento antes del reenvío.

Este tipo de organización de clases desacopla completamente la interfaz de la implementación de una abstracción y permite que las clases en las jerarquías de interfaz e implementación varíen sin afectarse mutuamente.

2. El Principio Abierto-Cerrado

El principio abierto-cerrado establece que un módulo de software debe ser:

- Abierto para la extensión — Debe ser posible alterar el comportamiento de un módulo o agregar nuevas características a su funcionalidad.
- Cerrado para la modificación — Dicho módulo no debe permitir que su código sea modificado.

En resumen, el principio abierto-cerrado ayuda a diseñar módulos de software cuya funcionalidad puede extenderse sin necesidad de modificar el código existente. Tener que modificar el código existente para extender la funcionalidad no es recomendable y viola este principio fundamental del diseño orientado a objetos.

En el contexto del registro de mensajes, si se modifica la interfaz Java para un tipo diferente de interfaz de aplicación, cada una de sus implementaciones necesita ser modificada, haciendo que la interfaz de la abstracción del logger y su implementación sean dependientes entre sí. Subclasificar la jerarquía de clases por cada tipo diferente de formato de mensaje tampoco es recomendable, ya que podría resultar en un número exponencial de subclases. El Patrón Puente se aplica en este caso para proveer la capacidad de agregar nuevos formatos de mensaje y nuevos tipos de implementaciones a la abstracción del logger.

3. Ejemplo — Abstracción de Registro de Mensajes

Se diseñó la funcionalidad de registro de mensajes para una aplicación durante la discusión del patrón Factory Method. El registro puede usarse para diferentes propósitos en distintas etapas de una aplicación, por lo que muchos objetos que forman parte de la aplicación pueden necesitar tener la capacidad de registrar mensajes. En adelante se usará la expresión abstracción logger para referirse a la abstracción de la funcionalidad de registro de mensajes.

Un mensaje puede registrarse en diferentes tipos de destinos, como un archivo, una consola y otros. Dependiendo del tipo de destino, se necesita una implementación diferente de la abstracción logger. Este requerimiento puede diseñarse con una interfaz Logger común que declara la interfaz (métodos) de la abstracción, y diferentes implementadores correspondientes a diferentes tipos de destino. Aplicando el Patrón Puente, la interfaz y las implementaciones de la abstracción logger se organizan en dos jerarquías de clases separadas.

3.1 Diseño de la Implementación de la Abstracción

Los implementadores de la abstracción logger deben proveer la implementación real necesaria para registrar mensajes en diferentes tipos de destino. Se definen dos implementadores — `FileLogger` y `ConsoleLogger` — para registrar mensajes en un archivo y en la consola, respectivamente. Estos implementadores se diseñan como implementadores concretos de una interfaz Java común `MessageLogger`.

La interfaz común `MessageLogger` declara un método `logMsg(String msg)`, que puede ser utilizado por los objetos que representan la interfaz de la abstracción logger.

Listado 3.1 — Interfaz `MessageLogger`

```
public interface MessageLogger {  
    public void logMsg(String msg);  
}
```

Listado 3.2 — Implementadores de `MessageLogger`: `FileLogger` y `ConsoleLogger`

```
public class FileLogger implements MessageLogger {  
    public void logMsg(String msg) {  
        FileUtil futil = new FileUtil();  
        futil.writeToFile("log.txt", msg, true, true);  
    }  
}  
  
public class ConsoleLogger implements MessageLogger {  
    public void logMsg(String msg) {  
        System.out.println(msg);  
    }  
}
```

Como parte de la implementación del método `logMsg`: el `FileLogger` escribe un mensaje dado en un archivo de log usando la clase `FileUtil`; el `ConsoleLogger` escribe el mensaje en pantalla. Los objetos cliente no deben acceder directamente al servicio de registro de mensajes ofrecido por los diferentes implementadores de la abstracción logger. Para registrar un mensaje, los objetos cliente interactúan con instancias de las clases que representan la interfaz de la abstracción logger, y estos objetos de interfaz a su vez utilizan los servicios de las clases implementadoras.

3.2 Diseño de la Interfaz de la Abstracción

La interfaz de la abstracción logger puede diseñarse en forma de clases que representan diferentes tipos de mensajes que un objeto cliente desearía registrar. Estas clases pueden diseñarse como implementadoras de una interfaz Java común.

La interfaz `Message` declara un método `log(String msg)` que puede ser utilizado por diferentes objetos cliente para registrar mensajes.

Se definen dos clases de interfaz de abstracción — `TextMessage` y `EncryptedMessage` — que representan un mensaje de texto plano y un mensaje encriptado, respectivamente. Estas clases se diseñan como implementadoras concretas de la interfaz Java común `Message`.

Listado 3.3 — Interfaz Message

```
public interface Message {  
    public void log(String msg);  
}
```

Listado 3.4 — Implementadores de Message: TextMessage y EncryptedMessage

```
public class TextMessage implements Message {  
    private MessageLogger logger;  
  
    public TextMessage(MessageLogger l) {  
        logger = l;  
    }  
  
    public void log(String msg) {  
        String str = preProcess(msg);  
        logger.logMsg(str);  
    }  
  
    private String preProcess(String msg) {  
        return msg;  
    }  
}  
  
public class EncryptedMessage implements Message {  
    private MessageLogger logger;  
  
    public EncryptedMessage(MessageLogger l) {  
        logger = l;  
    }  
  
    public void log(String msg) {  
        String str = preProcess(msg);  
        logger.logMsg(str);  
    }  
  
    private String preProcess(String msg) {  
        msg = msg.substring(msg.length() - 1)  
            + msg.substring(0, msg.length() - 1);  
        return msg;  
    }  
}
```

4. Aspectos Destacados del Diseño de las Clases de Interfaz

Tabla 1 — Características Clave del Diseño de las Clases de Interfaz de Abstracción

Característica de Diseño	Descripción
Separación de responsabilidades	Las clases de interfaz de abstracción (TextMessage, EncryptedMessage) no proveen implementación del servicio de registro. Eso es responsabilidad de los implementadores (FileLogger, ConsoleLogger).
Acceso indirecto al implementador	Los objetos cliente no acceden directamente a la interfaz expuesta por las clases implementadoras de la abstracción.
Referencia al implementador	Cada clase de interfaz de abstracción mantiene una referencia de objeto de tipo MessageLogger (implementador). Al crearse el objeto de interfaz, el cliente lo configura con un objeto MessageLogger.
Preprocesamiento privado	Al invocar el método log, el objeto de interfaz realiza el preprocesamiento requerido antes de delegar al implementador. El método preProcess es privado para evitar su uso externo.
Crecimiento lineal de clases	Agregar un nuevo formato de mensaje (ej. HTMLMessage) o un nuevo destino de log (ej. DBLogger) no requiere modificar las clases existentes, manteniendo el crecimiento de clases lineal.

5. Clase Cliente

Siempre que un cliente necesite registrar un mensaje, debe:

3. Crear una instancia de una clase implementadora de MessageLogger apropiada, como FileLogger o ConsoleLogger.
4. Crear una instancia de un implementador de Message apropiado, como TextMessage o EncryptedMessage.
5. Configurar el objeto implementador de Message con el objeto MessageLogger creado en el Paso 1. Este objeto se mantiene referenciado dentro del objeto implementador de Message.
6. Invocar el método log(String) sobre el objeto implementador de Message creado en el Paso 2.
7. El objeto implementador de Message realiza el preprocesamiento necesario para transformar el mensaje entrante al formato deseado (por ejemplo, encriptación en el caso de EncryptedMessage) y reenvía el mensaje transformado al objeto implementador de MessageLogger que contiene, invocando su método logMsg(String). Esta relación entre las clases en la jerarquía de interfaz y la jerarquía de implementadores constituye el Puente en este caso.

```

public class Client {
    public static void main(String[] args) {
        // Crear el objeto implementador apropiado
        MessageLogger logger = new FileLogger();

        // Elegir el objeto de interfaz requerido y
        // configurarlo con el objeto implementador
        Message msg = new EncryptedMessage(logger);
        msg.log("Mensaje de Prueba");
    }
}

```

La separación de la interfaz de la abstracción logger de su implementación permite que ambas sean modificadas de manera independiente sin necesidad de modificar la otra. Si un objeto cliente necesita registrar mensajes en un nuevo formato, por ejemplo HTML, este requerimiento puede atenderse fácilmente agregando una nueva clase HTMLMessage como implementadora de la interfaz Message existente. La adición de una nueva clase de interfaz no afecta ningún implementador de abstracción existente. De manera similar, una nueva implementación como un DBLogger para registrar mensajes en una base de datos puede agregarse sin modificar ninguna de las clases existentes en la jerarquía.

6. Patrón Puente versus Patrón Adaptador

Tabla 2 — Comparación entre el Patrón Puente y el Patrón Adaptador

Aspecto	Patrón Adaptador	Patrón Puente
Objetivo	Hacer que clases con interfaces incompatibles trabajen juntas.	Separar la interfaz de una abstracción de su implementación para que ambas puedan variar independientemente.
Momento de aplicación	Posterior al diseño inicial del sistema (patrón de adaptación posterior).	Durante el diseño del sistema, antes de la implementación (patrón de tiempo de diseño).
Interfaces incompatibles	El problema de las interfaces incompatibles es su razón de ser.	No existe el problema de interfaces incompatibles; los clientes siempre usan la interfaz expuesta por las clases de interfaz de abstracción.
Clase adaptado / implementador	Requiere una clase adaptado (existente) cuya interfaz se desea cambiar.	Define su propia jerarquía de implementadores desde el diseño.
Ocultamiento al cliente	Oculta los detalles de la implementación subyacente al cliente.	Oculta los detalles de la implementación subyacente al cliente.

En síntesis, aunque ambos patrones ocultan los detalles de la implementación subyacente al cliente, se aplican en contextos diferentes: el Patrón Adaptador resuelve problemas de compatibilidad entre interfaces existentes incompatibles, mientras que el Patrón Puente es una decisión proactiva de diseño para desacoplar interfaz e implementación desde el inicio, permitiendo que ambas evolucionen de forma independiente.

7. Preguntas de Práctica

8. Diseñe una aplicación que lea y escriba diferentes tipos de datos (texto plano, binario, etc.) hacia y desde diferentes destinos, como un archivo, una URL o una base de datos. Aplique el Patrón Puente en el diseño de la abstracción de lectura/escritura de datos.
9. Muchas aplicaciones con un backend de base de datos utilizan controladores ODBC/JDBC de diferentes proveedores. Identifique cómo se aplica el Patrón Puente cuando una aplicación utiliza un controlador ODBC/JDBC.
10. Diseñe una aplicación de formateo de código usando el Patrón Puente. Los programas pueden escribirse en cualquier lenguaje de programación (por ejemplo, Java, C++, etc.) y un programa dado puede formatearse de diferentes maneras, como formateo en blanco y negro, formateo HTML, formateo con colores y otros. La interfaz para el formateo de código puede implementarse de muchas maneras diferentes. Aplique el Patrón Puente para separar la interfaz de su implementación.